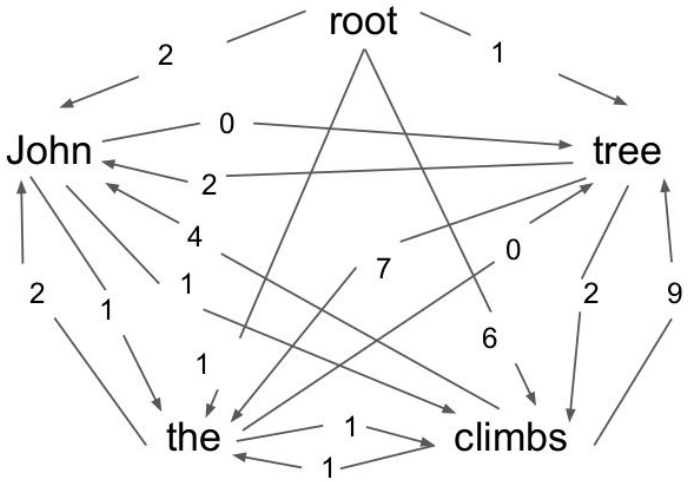
MST for Dependency Parsing

Finding the maximum spanning tree

Consider the sentence :

John climbs the tree

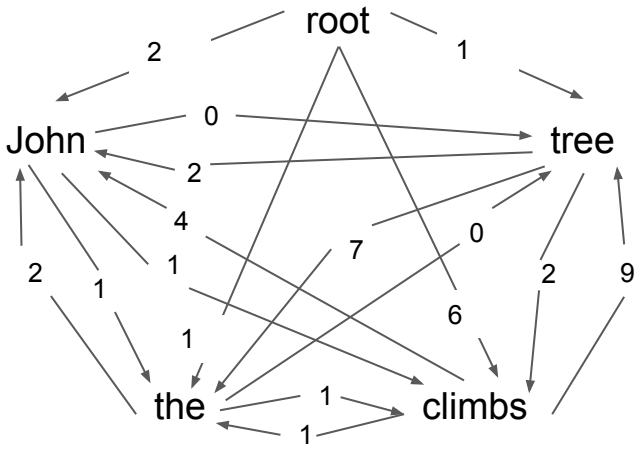
1. We generate all possible parses for the sentence and use a model to score all the edges*.
2. Find the MST using CLE (Chu-Liu Edmonds Algorithm)



*We have ignored edge labels to keep the example readable.

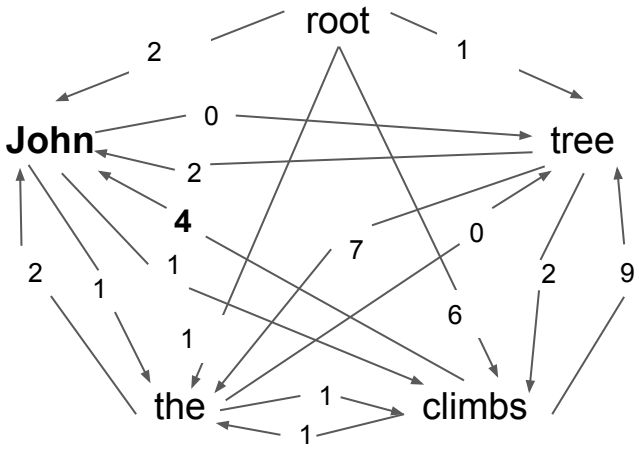
CLE Algorithm: *MST(G=(V,E), root, score)*

1. **Pick the highest incoming edge for each node in V**
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. **If** no cycles:
4. 0-edges constitute the MST
5. **Else**:
6. C ⇐ cycle
7. G' ⇐ Contract(G,C)
8. T' ⇐ MST(G', root, scores')
9. T ⇐ Expand(T', C)



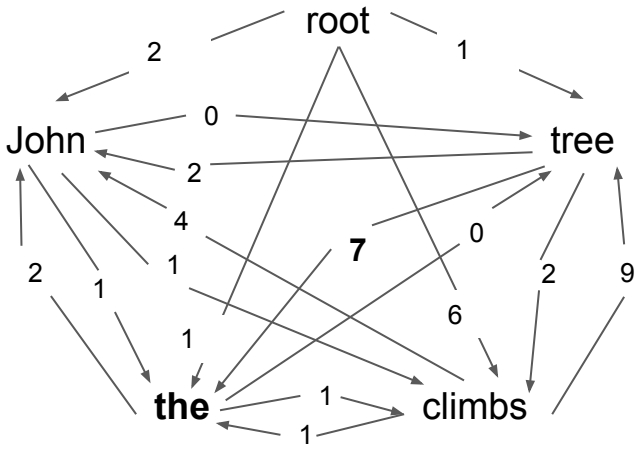
CLE Algorithm: *MST(G=(V,E), root, score)*

1. **Pick the highest incoming edge for each node in V**
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. **If** no cycles:
4. 0-edges constitute the MST
5. **Else**:
6. C ⇐ cycle
7. G' ⇐ Contract(G,C)
8. T' ⇐ MST(G', root, scores')
9. T ⇐ Expand(T', C)



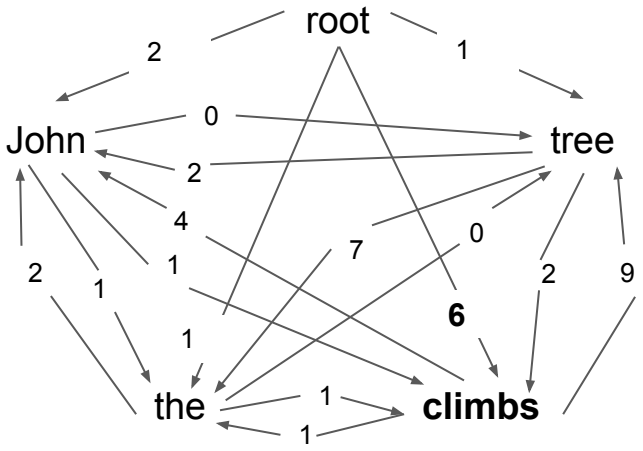
CLE Algorithm: *MST(G=(V,E), root, score)*

1. **Pick the highest incoming edge for each node in V**
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. **If** no cycles:
4. 0-edges constitute the MST
5. **Else**:
6. C ⇐ cycle
7. G' ⇐ Contract(G,C)
8. T' ⇐ MST(G', root, scores')
9. T ⇐ Expand(T', C)



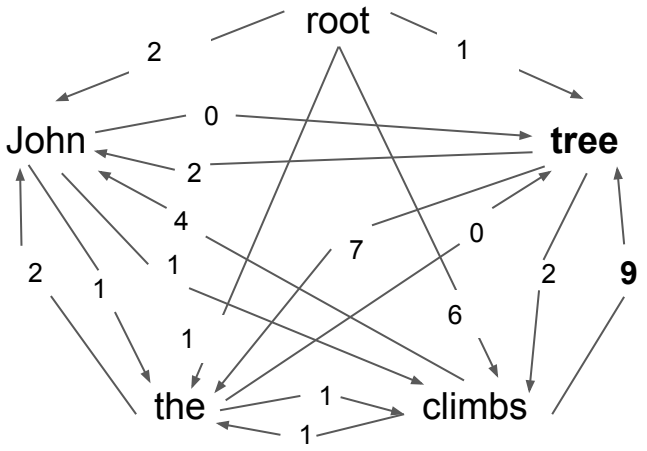
CLE Algorithm: *MST(G=(V,E), root, score)*

1. **Pick the highest incoming edge for each node in V**
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. **If** no cycles:
4. 0-edges constitute the MST
5. **Else**:
6. C ⇐ cycle
7. G' ⇐ Contract(G,C)
8. T' ⇐ MST(G', root, scores')
9. T ⇐ Expand(T', C)



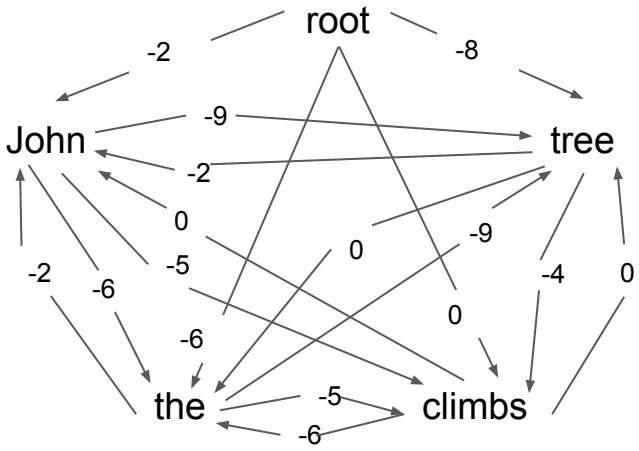
CLE Algorithm: *MST(G=(V,E), root, score)*

1. **Pick the highest incoming edge for each node in V**
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. **If** no cycles:
4. 0-edges constitute the MST
5. **Else**:
6. C ⇐ cycle
7. G' ⇐ Contract(G,C)
8. T' ⇐ MST(G', root, scores')
9. T ⇐ Expand(T', C)



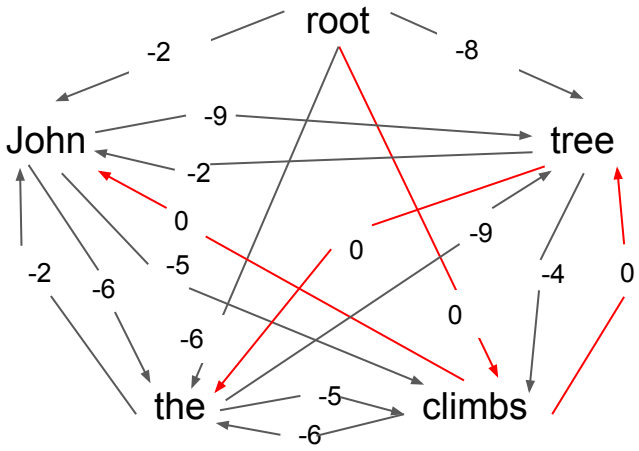
CLE Algorithm: *MST(G=(V,E), root, score)*

1. Pick the highest incoming edge for each node in V
2. **scores' ⇐ Subtract best edge scores from all incoming edges**
3. **If** no cycles:
4. 0-edges constitute the MST
5. **Else**:
6. C ⇐ cycle
7. G' ⇐ Contract(G,C)
8. T' ⇐ MST(G', root, scores')
9. T ⇐ Expand(T', C)



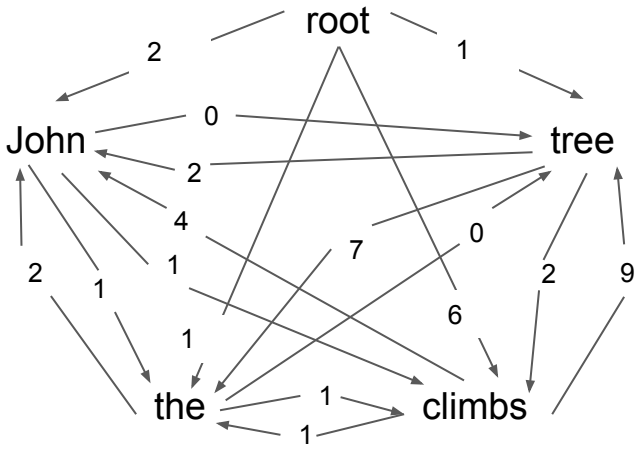
CLE Algorithm: *MST(G=(V,E), root, score)*

1. Pick the highest incoming edge for each node in V
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. **If no cycles:**
4. **0-edges constitute the MST** ✓
5. **Else**:
6. C ⇐ cycle
7. G' ⇐ Contract(G,C)
8. T' ⇐ MST(G', root, scores')
9. T ⇐ Expand(T', C)



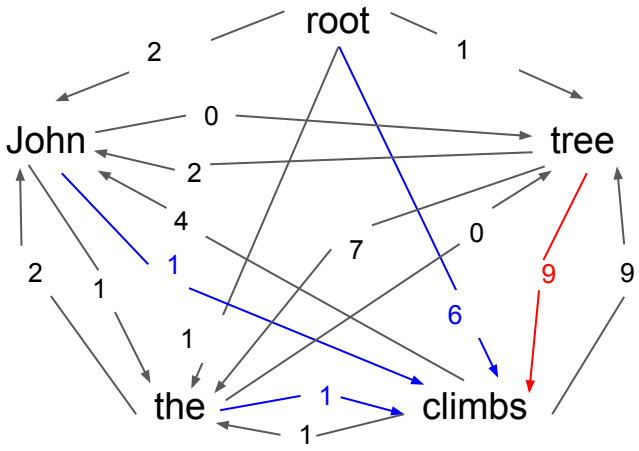
What about cycles?

1. Pick the highest incoming edge for each node in V
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. **Else:**
6. **C ⇐ cycle**
7. **G' ⇐ Contract(G,C)**
8. **T' ⇐ MST(G', root, scores')**
9. **T ⇐ Expand(T', C)**



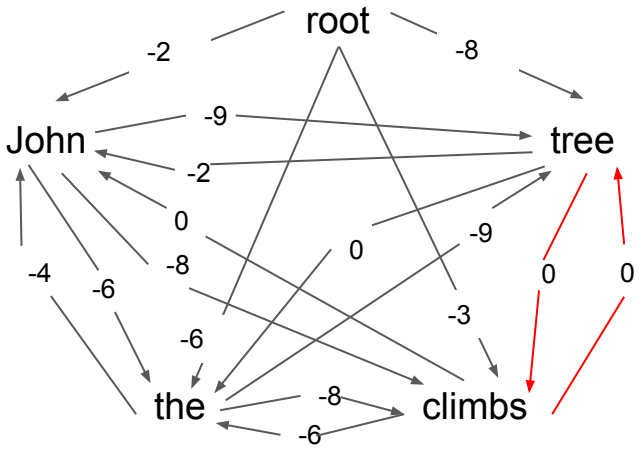
Let’s make a cycle to see

1. Pick the highest incoming edge for each node in V
2. scores’ ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. Else:
6. C ⇐ cycle
7. G’ ⇐ Contract(G,C)
8. T’ ⇐ MST(G’, root, scores’)
9. T ⇐ Expand(T’, C)



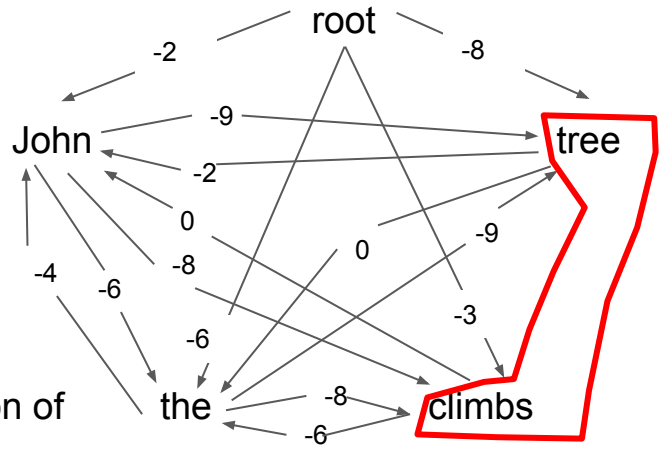
Let’s make a cycle to see

1. Pick the highest incoming edge for each node in V
2. scores’ ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. Else:
6. **C ⇐ cycle**
7. G’ ⇐ Contract(G,C)
8. T’ ⇐ MST(G’, root, scores’)
9. T ⇐ Expand(T’, C)



So how do we contract?

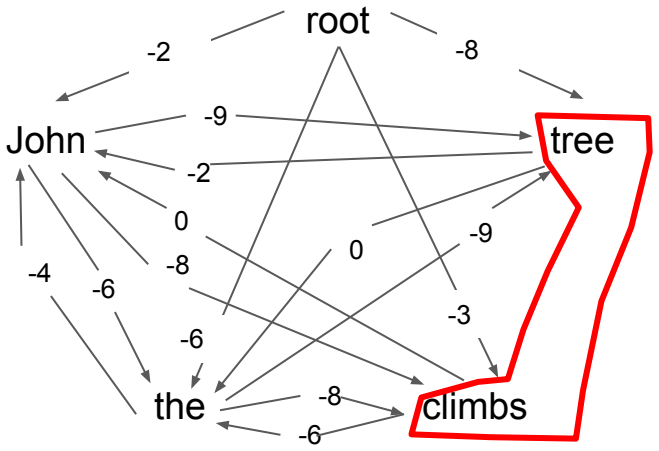
1. Pick the highest incoming edge for each node in V
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. Else:
6. C ⇐ cycle
7. **G' ⇐ Contract(G,C)**
8. T' ⇐ MST(G', root, scores')
9. T ⇐ Expand(T', C)



⇒ We construct a contracted node C that has the union of all incoming and outgoing edges of *tree* and *climbs*

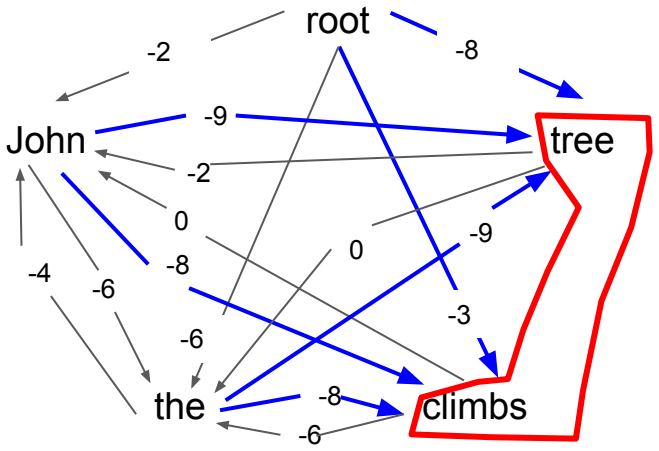
Find the MST for this new graph G’

1. Pick the highest incoming edge for each node in V
2. scores’ ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. Else:
6. C ⇐ cycle
7. G’ ⇐ Contract(G,C)
8. **T’ ⇐ MST(G’, root, scores’)**
9. T ⇐ Expand(T’, C)



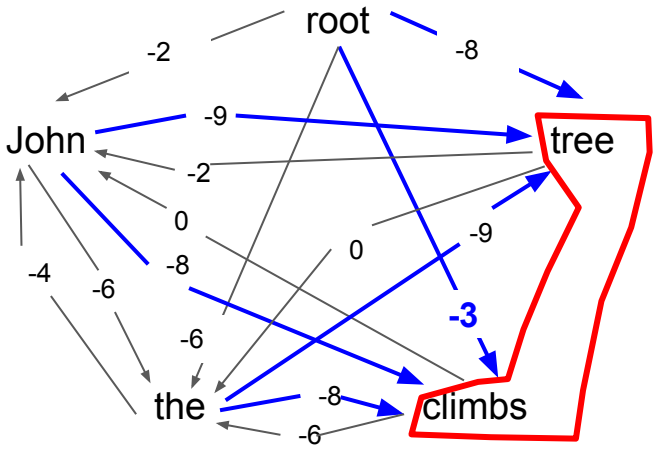
Find the MST for this new graph G’

1. Pick the highest incoming edge for each node in V
2. scores’ ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. Else:
6. C ⇐ cycle
7. G’ ⇐ Contract(G,C)
8. **T’ ⇐ MST(G’, root, scores’)**
9. T ⇐ Expand(T’, C)



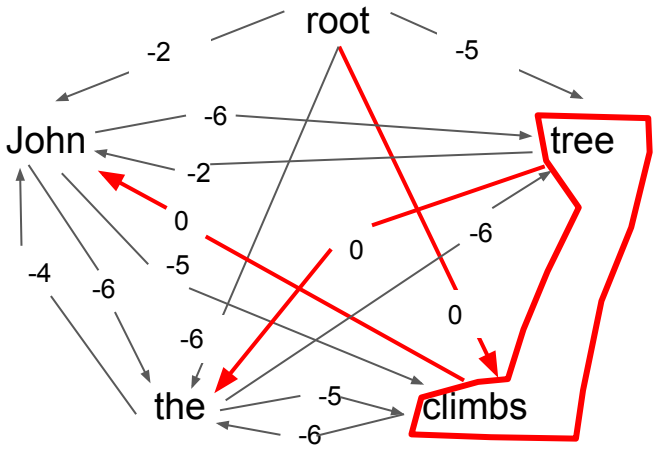
Find the MST for this new graph G’

1. Pick the highest incoming edge for each node in V
2. scores’ ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. Else:
6. C ⇐ cycle
7. G’ ⇐ Contract(G,C)
8. **T’ ⇐ MST(G’, root, scores’)**
9. T ⇐ Expand(T’, C)



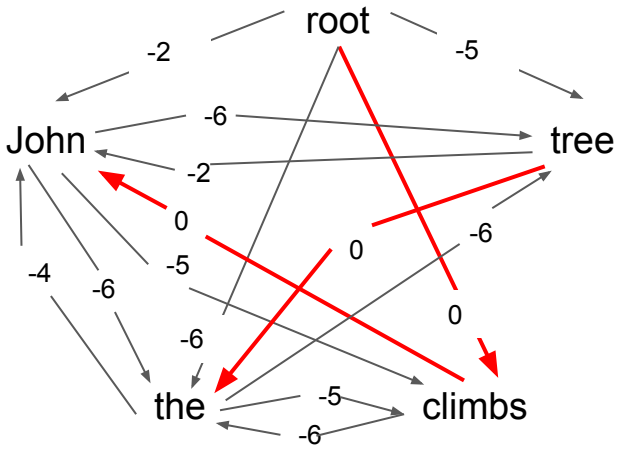
Find the MST for this new graph G’

1. Pick the highest incoming edge for each node in V
2. scores’ ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. Else:
6. C ⇐ cycle
7. G’ ⇐ Contract(G,C)
8. **T’ ⇐ MST(G’, root, scores’)**
9. T ⇐ Expand(T’, C)



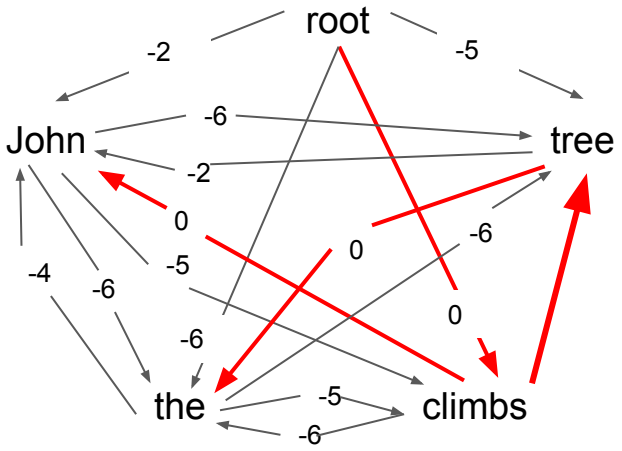
Expand the cycle

1. Pick the highest incoming edge for each node in V
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. Else:
6. C ⇐ cycle
7. G' ⇐ Contract(G,C)
8. T' ⇐ MST(G', root, scores')
9. **T ⇐ Expand(T', C)**



Expand the cycle ✓

1. Pick the highest incoming edge for each node in V
2. scores' ⇐ Subtract best edge scores from all incoming edges
3. If no cycles:
4. 0-edges constitute the MST
5. Else:
6. C ⇐ cycle
7. G' ⇐ Contract(G,C)
8. T' ⇐ MST(G', root, scores')
9. **T ⇐ Expand(T', C)**



⇒ Only one possibility, as *climbs* now has an incoming edge but *tree* does not